

3.vii Punteros

- **Puntero:** variable que contiene una dirección de memoria, normalmente la posición de otra variable. Si una variable contiene la dirección de otra variable se dice que la primera apunta a la segunda.
- El uso de **punteros en C** aporta:
 - Permiten **modificar los argumentos de las funciones**.
 - Mejoran la **eficiencia de las funciones**.
 - Permiten el uso de **memoria dinámica**.
 - Se usan como **soporte para estructuras de datos**: listas, arboles, etc.
- **Declaración:**
*tipo *nombre;*
C asume que a lo que apunta un puntero es un objeto de su tipo base.
- Existen dos **operadores especiales de punteros**: **&** y *****.
 - **&**: devuelve la dirección de memoria de su operando. Por ejemplo:
*int cuenta, *m;*
m=&cuenta;
 - *****: devuelve el valor de la variable localizada en la dirección que sigue.

3.vii Punteros

```
/*Programa que ilustra el valor 100 en pantalla*/
#include <stdio.h>
int main()
{
    int cuenta, q;
    int *m=NULL;

    cuenta=100; /* a cuenta se le asigna 100 */
    m = &cuenta; /* m recibe la dirección de cuenta */
    q = *m; /* a q se le asigna directamente el valor de cuenta
              indirectamente a través de m */
    printf("%d", q); /* imprime 100 */

    return 0;
}
```

3.vii Punteros

```
/*Programa en el que se pierde la parte decimal de un float*/  
#include <stdio.h>  
int main( )  
{  
    float x,y;  
    int *p=NULL;  
  
    x = 100.123;  
    p=&x;  
    y=*p; /* p toma valor entero */  
    printf("%f", y); /* esto funcionará mal */  
  
    return 0;  
}
```

3.vii Punteros

Vectores y punteros:

En C los vectores y los punteros están estrechamente relacionados. Los **vectores** son zonas de memoria que contienen un número de elementos de un determinado tipo de datos. Mientras que un **puntero** apunta al comienzo de una zona de memoria que contiene datos.

Cuando se declara **char p[10];** se está declarando una zona de datos para 10 caracteres. p indica el puntero al comienzo de la misma. Luego **p es igual a &p[0]**.

Cualquier variable puntero puede “*indexarse*” como si estuviera declarada como un vector del tipo base del puntero.

```
int *p, i[10];  
p=i;  
p[5]=100; /* Asignación con índice */  
*(p+5)=100; /*Asignación */
```

```
/*Imprimir una cadena al revés*/  
  
#include <malloc.h>  
#include <stdio.h>  
#include <string.h>  
  
int main()  
{  
    char *c;  
    int t;  
  
    c=(char *) malloc(80);  
    if (c==0)  
    {  
        printf("fallo de memoria\n");  
        return -1;  
    }  
  
    printf("Dame una cadena.\n");  
    gets(c); /*Lee un vector de caracteres*/  
  
    for(t=strlen(c)-1;t>=0;t--)  
        printf("%c",c[t]);  
    free(c);  
    return 0;  
}
```

3.vii Punteros

- **Asignaciones de punteros:** Un puntero puede usarse para apuntar a otro.

```
int x, *p1, *p2;
```

```
p1=&x;
```

```
p2=p1;
```

- **Asignación de memoria dinámica:** C permite reservar y liberar memoria en tiempo de ejecución. Funciones:

```
void malloc(size_t num_bytes);
```

```
void free(void *p);
```

- **malloc:** asigna memoria y devuelve un puntero *void* al comienzo de ella.
- **free:** libera la memoria previamente asignada por el programa.
- **sizeof:** calcula el nº de bytes del tipo de datos o variable pasado como parámetro.

```
#include<stdio.h>
#include<malloc.h>
#include<string.h>
int main()
{
    char *p;
    int n;

    printf("Dame el numero de
           casillas\n");
    scanf("%d",&n);
    p=(char *)malloc(sizeof(char)*n);
    strcpy(p,"cadena de
           caracteres\n");
    printf("%s",p);

    free(p);

    return 0;
}
```

3.viii Funciones

Características de las funciones dividen los programas:

- Las funciones **deben estar en el mismo programa** que la función principal. También puede **incluirse un fichero que las contenga con *#include*** y enlazarse posteriormente con el enlazador.
- No se permite **anidamiento de funciones**.
- Pueden ser **invocadas o llamadas** por **otras funciones**, por el **programa principal** o **por sí mismas** (recursividad).
- **Pueden tener o no argumentos**, también llamados parámetros.
- **Pueden devolver un valor o no**.
- Pueden **poseer sus propias variables**.
- Tienen **un punto de entrada** y pueden tener **varios de salida** (sentencia *return*).
- **En un programa puede haber cualquier número de funciones**. Al menos debe haber una, llamada *main*.

3.viii Funciones

■ Forma general de una función

```
especificador_tipo nombre(lista_parámetros)
{
    cuerpo_función
}
```

- *especificador_tipo*: Tipo de valor que devuelve la función. Si no se especifica nada, el compilador asume que la función devuelve un entero.
- *nombre*: Nombre de la función.
- *lista_parámetros*: Lista de variables que van a recibir los valores de los argumentos actuales; se separan por comas. Los paréntesis se requieren siempre, aunque una función no tenga parámetros.

■ Lista de parámetros

```
tipo nombre(tipo1 par1, tipo2 par2, ...)
```

```
{
    cuerpo;
}
```

- En las funciones los parámetros deben declararse individualmente.
- Los parámetros formales tienen que tener el mismo tipo que los parámetros actuales.

3.viii Funciones

■ Constantes y variables locales

- En C cada función es un bloque de código.
- El código y los datos que están definidos dentro de una función no pueden interactuar con el código o los datos definidos dentro de otra función o el programa principal.
- Las variables que están definidas dentro de una función se llaman **variables locales**. Una variable local comienza a existir cuando se entra en la función y se destruye al salir de ella. Así, las variables locales no pueden conservar sus valores entre distintas llamadas a la función.
- De la misma manera que las variables locales, se pueden definir **constantes locales** que serán visibles sólo dentro de la función.

```
char minusc_a_mayusc(char c1)
{
    char c2;

    if (c1>='a' && c1<='z') c2='A'+c1-'a';
    else c2=c1;

    return c2;
}
```

```
long int factorial(int n)
{
    int i;
    long int prod=1;
    if(n>1)
        for(i=2; i<=n; i++)
            prod=prod*i;
    return prod;
}
```


3.viii Funciones

Invocación de funciones

- Invocación o llamada a una función: Especificando su nombre, seguido de una lista de argumentos encerrados entre paréntesis y separados por comas. Si no requiere ningún argumento, se debe escribir a continuación del nombre de la función un par de paréntesis.
- La **llamada a la función puede formar parte de una expresión simple** (como por ejemplo una instrucción de asignación) o **puede ser uno de los operandos de una expresión más compleja**.
- Los argumentos que aparecen en la llamada a la función se denominan **parámetros actuales**, en contraste con los **parámetros formales** que aparecen en la primera línea de la definición de una función (también son llamados argumentos actuales o formales).
- Los **argumentos actuales** pueden ser constantes, variables simples o expresiones más complejas. Cada argumento actual debe ser del mismo tipo de datos que el argumento formal correspondiente.

```
#include <stdio.h>
char minusc_a_mayusc(char c1)
{
    char c2;

    if(c1>='a' && c1<='z') c2='A'+c1-'a';
    else c2=c1;
    return c2;
}

int main( )
{
    char minusc,mayusc;

    printf("Introduce una letra minúscula:");
    scanf("%c",&minusc);
    mayusc=minusc_a_mayusc(minusc);
    printf("La mayúscula es: c\n",mayusc);
}
```

3.viii Funciones

Paso de parámetros a funciones

- Si una función va a usar argumentos, debe declarar variables que acepten los valores de los argumentos.
- Estas variables se comportan como otras variables locales dentro de la función, creándose al entrar en la función y destruyéndose al salir.
- Se pueden pasar argumentos a las funciones de dos formas: **llamada por valor** y **llamada por referencia**.
- El método de **llamada por valor** copia el valor de un argumento en el parámetro formal de la función. En este caso, **los cambios en el parámetro formal no afectan al argumento original**.
- La **llamada por referencia** copia la dirección del argumento en el parámetro. Dentro de la función se usa el puntero al argumento usado en la llamada. Esto significa que **los cambios sufridos por el parámetro afectan al argumento**.

```
/*Paso por valor*/  
#include <stdio.h>  
int cuad(int x)  
{  
    x = x * x;  
    return x;  
}  
int main(void)  
{  
    int t=10;  
  
    printf("%d %d",cuad(t),t);  
    return 0;  
}
```

```
#include <stdio.h>
void interVALOR(int x, int y) /*Paso por valor*/
{
    int temp;

    temp = x;
    x = y;
    y = temp;
}
void interREF(int *x, int *y) /*Paso por referencia*/
{
    int temp;

    temp = *x;
    *x = *y;
    *y = temp;
}
int main(void)
{
    int i=10,j=20;

    printf("INICIALES: i=%d,j=%d\n",i,j);
    interVALOR(i,j); /*paso de las variables i y j */
    printf("DESPUES DE LLAMADA VALOR: i=%d,j=%d\n",i,j);
    interREF(&i,&j); /*paso de las direcciones de i y j */
    printf("DESPUES DE LLAMADA REFERENCIA: i=%d,j=%d\n",i,j);
    return 0;
}
```

3.viii Funciones

La sentencia *return*

return [expresión];

- La sentencia *return* se utiliza para:
 - Devolver un valor.
 - Para forzar la salida inmediata de la función en que se encuentra.
- Formas de terminar la función su ejecución y volver al sitio en que se invocó:
 - Cuando se ha ejecutado la última sentencia de la función y se encuentra la llave }.
 - Cuando se emplea la sentencia *return* para terminar la ejecución.
- Una función puede tener varias sentencias *return*.
- Si una función devuelve valores hay que declarar este tipo de devolución explícitamente.
- Todas las funciones, excepto aquellas de tipo *void*, devuelven un valor. Este valor se especifica explícitamente en la sentencia *return*.

```
#include <string.h>
#include <stdio.h>

void imp_inversa(char *s)
{
    register int t;

    for(t=strlen(s)-1; t>=0; t--)
        putchar(s[t]);
}

int main( )
{
    imp_inversa("Me gusta C");
    return 0;
}
```

```
int factorial(int n)
{
    if(n==0 || n==1)
        return 1;
    else
        return n*factorial(n-1);
}
```

3.viii Funciones

Funciones de tipo void

- Uno de los usos de *void* es el de declarar funciones que no devuelven valores.
- Una función de tipo *void* no puede utilizarse como operando en cualquier expresión.
- Tampoco puede una función ser el destino de una asignación.

```
#include <stdio.h>

void *encuentra(char c, char *s)
{
    while(c!=*s && *s) s++;
    return s;
}

void main(void)
{
    char s[80],*p,c;

    gets(s);
    c=getchar();
    p=encuentra(c,s);
    if(p) /* se ha encontrado */
        printf("%s",p);
    else
        printf("No se ha encontrado");
}
```

3.viii Funciones

Paso de vectores a funciones

- | En C no se puede pasar un **vector completo** como argumento a una función.
- | Se pasa el puntero al vector.
- | El **resultado de los tres métodos de declaración es el mismo** porque de las tres formas se le indica que se le va a pasar un puntero a entero.
- | C no comprueba los límites del vector.

```
main ( )
{
    int i[15];

    func1(i);
}

func1(int *x) /*puntero*/
{
}

func1(int x[10]) /*vector delimitado*/
{
}

func1(int x[]) /*vector no delimitado*/
{
}
```

3.viii Funciones

Paso de matrices a funciones:

- | Se debe definir al menos la longitud de la segunda dimensión, ya que el compilador necesita saber la dimensión de cada fila para indexar el vector correctamente. También se puede decir la longitud de la primera dimensión, pero no es necesario.

```
func1(int x[][10])  
{  
    instrucción 1;  
    instrucción 2;  
    instrucción N;  
}
```

Devolución de punteros

- | Las funciones que devuelven punteros se manejan de la misma forma que cualquier función.
- | Los punteros a variables son las direcciones de memoria de un determinado tipo de datos.
- | Para devolver un puntero, una función debe ser declarada para devolver un puntero: se incluirá un asterisco delante del nombre de la función.

3.viii Funciones

```
#include<stdio.h>
#include<malloc.h>
#include<string.h>

char *funcion(char v[ ],char w[ ])
{
    char *r;
    int tam;

    tam=strlen(v)+strlen(w)+1;
    printf("TAM=%d\n",tam);
    r=malloc(sizeof(char)*tam);
    strcpy(r,v);
    strcat(r,w);

    return r;
}
```

```
int main()
{
    char v[50],w[50],*t;
    int i;

    printf("Cadena de menos de 50 caracteres\n");
    gets(v);
    printf("Cadena de menos de 50 caracteres\n");
    gets(w);
    t=funcion(v,w);
    printf("CADENA= %s\n",t);
    free(t);

    return 0;
}
```


3.viii Funciones

```
#include<stdio.h>
#include<malloc.h>

int *reservar(int tam)
{
    return (int *)malloc(sizeof(int)*tam);
}

void liberar(int *v)
{
    free(v);
}

void rellenar(int *v,int tam)
{
    int i;

    for(i=0;i<tam;i++)
    {
        printf("Dame el valor de la celda %d\n",i);
        scanf("%d",&v[i]);
    }
}
```

```
void mostrar(int *v,int tam)
{
    int i;

    printf("[");
    for(i=0;i<tam;i++) {
        if (i<(tam-1)) printf("%d, ",v[i]);
        else printf("%d",v[i]);
    }
    printf("]\n");
}

int main()
{
    int n,*vect;

    printf("Dame el numero de celdas\n");
    scanf("%d",&n);
    vect=reservar(n);
    rellenar(vect,n);
    mostrar(vect,n);
    liberar(vect);
    return 0;
}
```